

L01, Group 06
Melody Yang 400307007
Cynthia Sun 400238765



MECHTRON 3TB4
Lab Tutorial 2 Report
Lab 2 Prelab Report

Date: February 13, 2026

Lab Tutorial 2 Report

Overview

In this lab tutorial, a stopwatch was designed and implemented on the DE1-SoC board with an FPGA. The goal of the lab was to gain experience creating a hardware interface using Verilog and handling multiple modules in a single design. To do this, a clock divider was used to reduce the 50 MHz system clock to 1 kHz, which would increment a counter every millisecond. The counter was then converted from binary to BCD and displayed on the seven-segment hex displays. The push buttons were used to reset, start/resume, and stop the stopwatch. The design was simulated and verified to ensure proper operation and then implemented on the FPGA.

Part 1: Binary to BCD Converter

The goal of the Binary to BCD (Binary-Coded Decimal) module is to convert a binary model into BCD format, allowing the value to be correctly displayed on the seven-segment display on the DE1-SoC board. The algorithm used is called “shift-add-3”, which involves using an iterative process to shift the bits and adjust the digit values that are greater than or equal to 5. Since the board can display multiple digits, the Binary to BCD module can convert several BCD digits simultaneously. This allows the numerical values to appear in decimal form, making it easier to read on the seven-segment display.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Wed Feb 11 23:25:44 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	tut2
Top-level Entity Name	hex_to_bcd_converter
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	91 / 32,070 (< 1 %)
Total registers	0
Total pins	46 / 457 (10 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 1. Compilation report for the Binary to BCD converter.

Table 1. Summary for Components for Binary to BCD Converter in Lab 2 Tutorial Part 1.

Feature	Quantity
Total number of logic elements	91
Total number of registers	0
Total number of pins	46
Max number of logic elements that can fit on FPGA	32,070

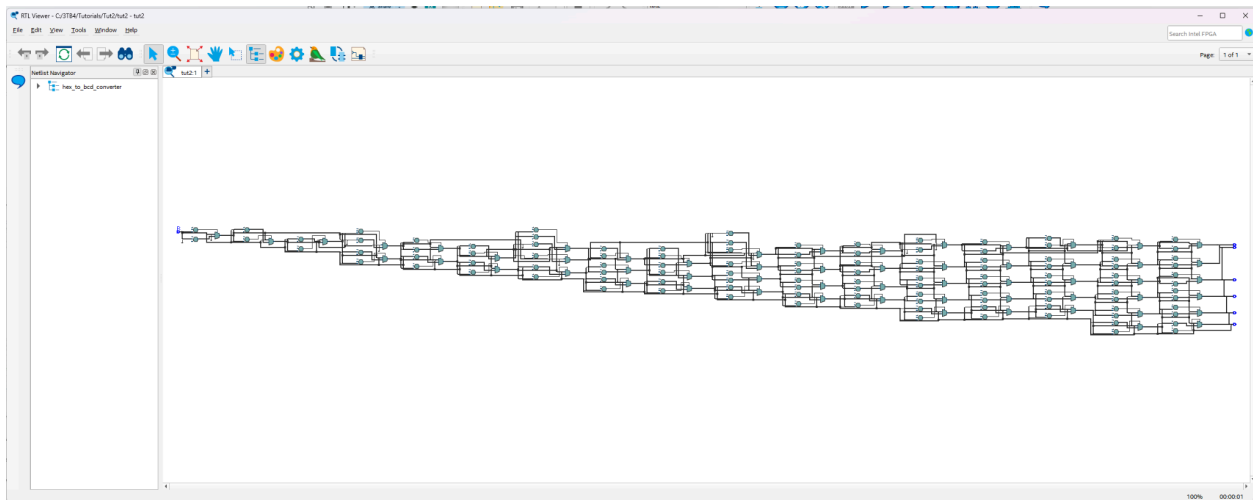


Figure 2. Screenshot of the implemented Binary to BCD converter.

```

1 // CONVERTS HEX TO BCD
2
3 module hex_to_bcd_converter(input wire clk, reset,
4                             input wire [19:0] hex_number,
5                             output [3:0] bcd_digit_0,bcd_digit_1,bcd_digit_2,bcd_digit_3, bcd_digit_4, bcd_digit_5);
6
7 // DE1-SoC has 6 7_seg_LEDs. 20 bits can represent decimal 999999.
8 //This module is designed to convert a 20 bit binary representation to BCD
9 integer i, k, j;
10 wire [19:0] hex_number1; // the last 20 bits of input hex_number
11 reg [3:0] bcd_digit [5:0]; // 6 BCD for 0-5
12 assign hex_number1 = hex_number[19:0];
13
14 assign bcd_digit_0 = bcd_digit[0];
15 assign bcd_digit_1 = bcd_digit[1];
16 assign bcd_digit_2 = bcd_digit[2];
17 assign bcd_digit_3 = bcd_digit[3];
18 assign bcd_digit_4 = bcd_digit[4];
19 assign bcd_digit_5 = bcd_digit[5];
20
21 always @ (*) begin
22     if (!reset) begin
23         //set all 6 digits to 0
24         for (j=5; j>=0; j=j-1) begin
25             bcd_digit[j] = 4'b0000;
26         end
27         //set all 6 digits to 0
28         for (j=5; j>=0; j=j-1) begin
29             bcd_digit[j] = 4'b0000;
30         end
31
32         //shift 20 times
33         for (i=19; i>=0; i=i-1)
34             begin
35
36                 //check all 6 BCD tetrads
37                 for (k=5; k>=0; k=k-1)
38                     begin
39                         // if >=5 then add 3
40                         if (bcd_digit[k] >= 5) begin
41                             bcd_digit[k] = bcd_digit[k] + 4'b0011;
42                         end
43                     end
44
45                 //shift one bit of BIN/HEX left
46                 //for the first 5 tetrads
47                 for (k=5; k>=1; k=k-1)
48                     begin
49                         bcd_digit[k]=bcd_digit[k] << 1;
50                         bcd_digit[k][0]=bcd_digit[k-1][3];
51                     end
52
53                 //shift one bit of BIN/HEX left, for the last tetrad
54                 /* fill your code here */
55                 bcd_digit[0] = bcd_digit[0] << 1;
56                 bcd_digit[0][0] = hex_number1[i];
57             end //end for loop
58         end //end of always.
59     endmodule
60
61

```

Figure 3a. Screenshot of the module of the Binary to BCD converter.

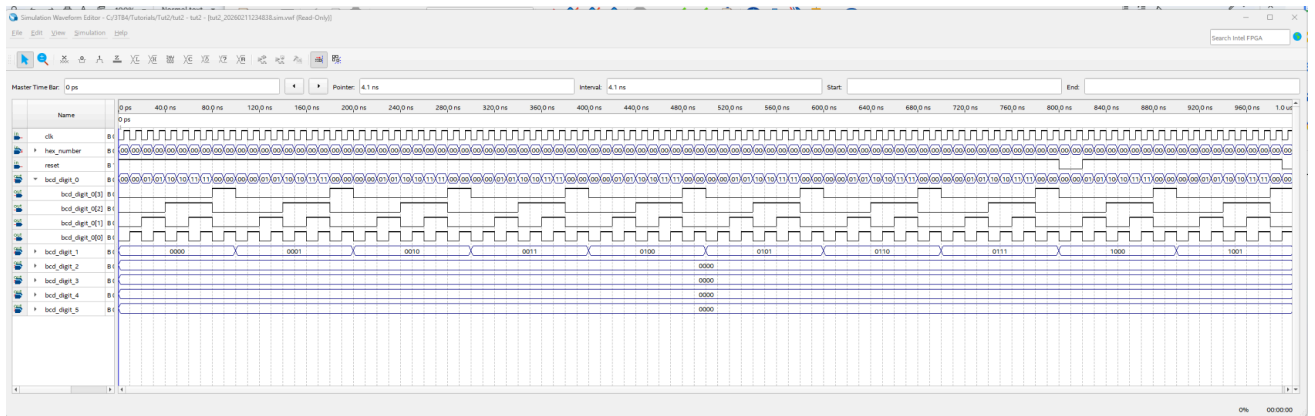


Figure 3b. Screenshot of the simulation of the Binary to BCD converter.

Part 2: Clock Divider

The goal of the Clock Divider Module is to reduce the frequency of the 50 MHz system clock on the DE1-SoC board to a lower frequency that is suitable for this lab. The native clock is too fast for humans to observe, and for this lab, where millisecond timing is required. This module uses a counter to toggle an output signal once after a specified counter threshold is achieved, effectively generating a lower clock frequency. The clock output for this module is used as the clock signal for other modules used in this lab.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Wed Feb 11 23:54:51 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	tut2
Top-level Entity Name	clock_divider
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	25 / 32,070 (< 1 %)
Total registers	44
Total pins	3 / 457 (< 1 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 4. Compilation report for Clock Divider.

Table 2. Summary for Components for Clock Divider in Lab 2 Tutorial Part 2.

Feature	Quantity
Total number of logic elements	25
Total number of registers	44
Total number of pins	3
Max number of logic elements that can fit on FPGA	32,070

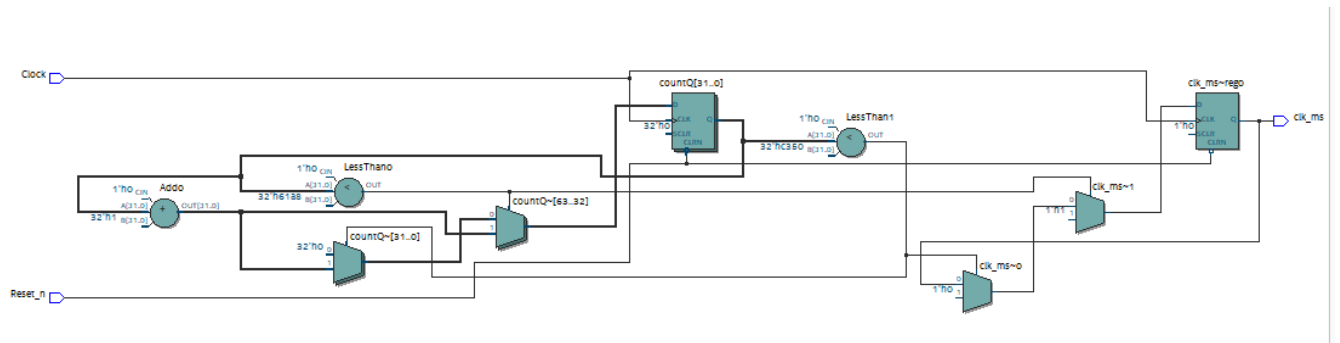


Figure 5. Screenshot of the implemented Clock Divider.

```

1 // CLOCK DIVIDER => MAKES FAST CLOCK (50 MHZ) --> SLOWER (1 KHZ)
2
3 module clock_divider (input clock, Reset_n, output reg clk_ms);
4     parameter factor = 50000; //50000; // 32'h000061a7;
5
6     reg [31:0] countQ;
7
8     always @ (posedge clock, negedge Reset_n) begin
9         if (!Reset_n) begin
10             countQ <= 32'b0; // reset countQ to 0
11             clk_ms <= 1'b0;
12         end
13         else begin
14             if (countQ < factor/2) begin // clk is low
15                 countQ <= countQ + 1'b1; // increment countQ
16                 clk_ms <= 1'b1;
17             end
18
19             else if (countQ < factor) begin // clk is high
20                 countQ <= countQ + 1'b1;
21                 clk_ms <= 1'b0;
22             end
23
24             else begin //countQ == factor => nxt clk cycle
25                 countQ <= 32'b0; // reset countQ
26             end
27         end
28     end
29 endmodule
30
31
32

```

Figure 6a. Screenshot of the module of the Clock Divider.

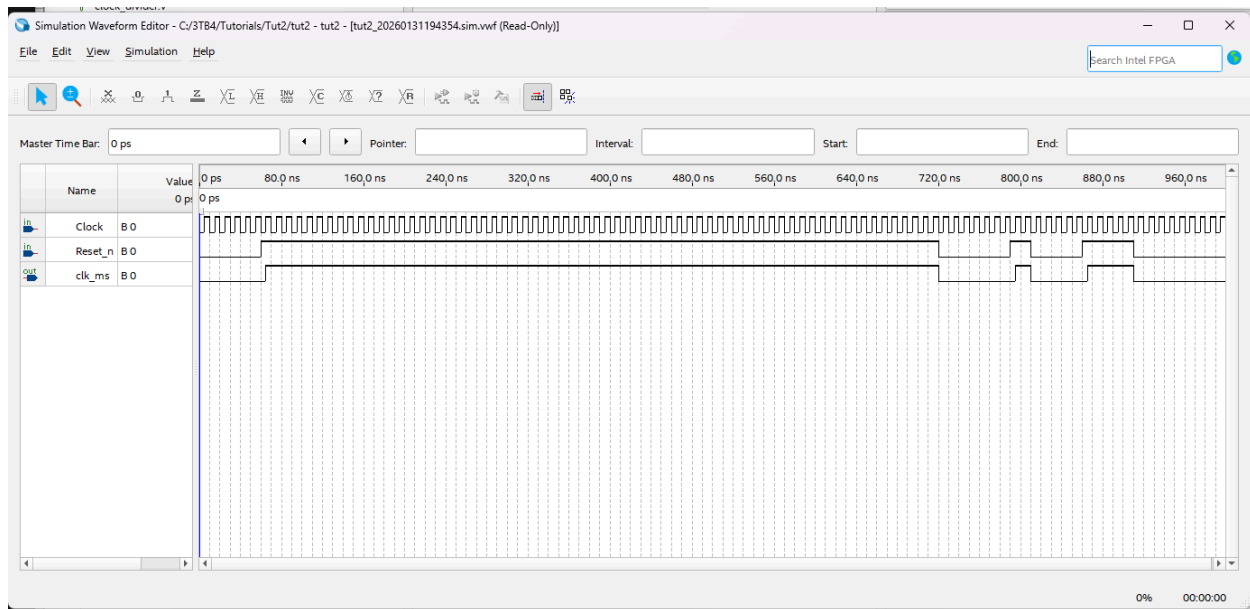


Figure 6b. Screenshot of the simulation of the Clock Divider.

Part 3: Counter

The goal of the counter module is to track elapsed time in milliseconds, and it uses the divided clock signal as its input. The user can reset, start, stop, and resume the counter depending on which KEY is pressed. When enabled, the counter increments on each clock pulse, and when stopped, it retains its current value. The output of the counter is used by other modules, such as displaying on the seven-segment display.

Flow Status	Successful - Thu Feb 12 00:00:11 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	tut2
Top-level Entity Name	counter
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	12 / 32,070 (< 1 %)
Total registers	21
Total pins	24 / 457 (5 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 7. Compilation report for the Counter.

Table 3. Summary for Components for Counter Lab 2 Tutorial Part 2.

Feature	Quantity
Total number of logic elements	12
Total number of registers	21
Total number of pins	24
Max number of logic elements that can fit on FPGA	32,070

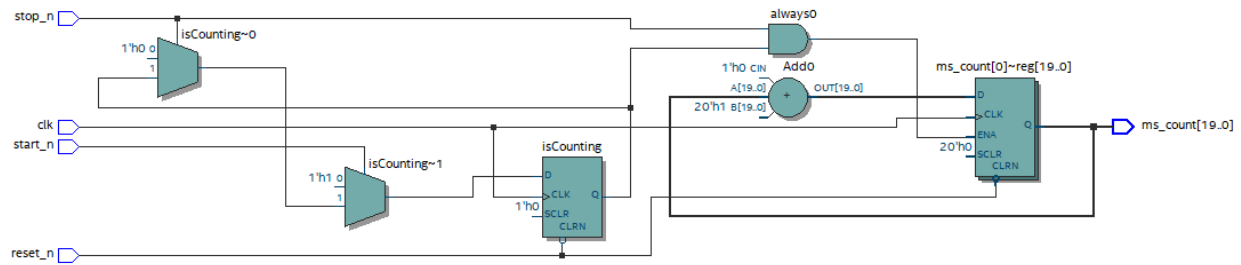


Figure 8. Screenshot of the implemented Counter.

```

1  // COUNTER
2
3  module counter(input clk, input reset_n, start_n, stop_n, output reg [19:0] ms_count);
4      reg isCounting; // counter flag
5
6      always @(posedge clk or negedge reset_n) begin
7          if (!reset_n) begin
8              ms_count <= 20'b0; // reset counter
9              isCounting <= 1'b0; // reset flag
10         end
11
12         else begin
13             if (!start_n) begin
14                 isCounting <= 1'b1; // start counting
15             end
16
17             else if (!stop_n) begin
18                 ms_count <= ms_count; // store prev value
19                 isCounting <= 1'b0; // stop counting
20             end
21
22             if (isCounting && stop_n) begin
23                 ms_count <= ms_count + 1'b1;
24             end
25
26         end
27     end
28 end
29
30 endmodule

```

Figure 9a. Screenshot of the module of the Counter.

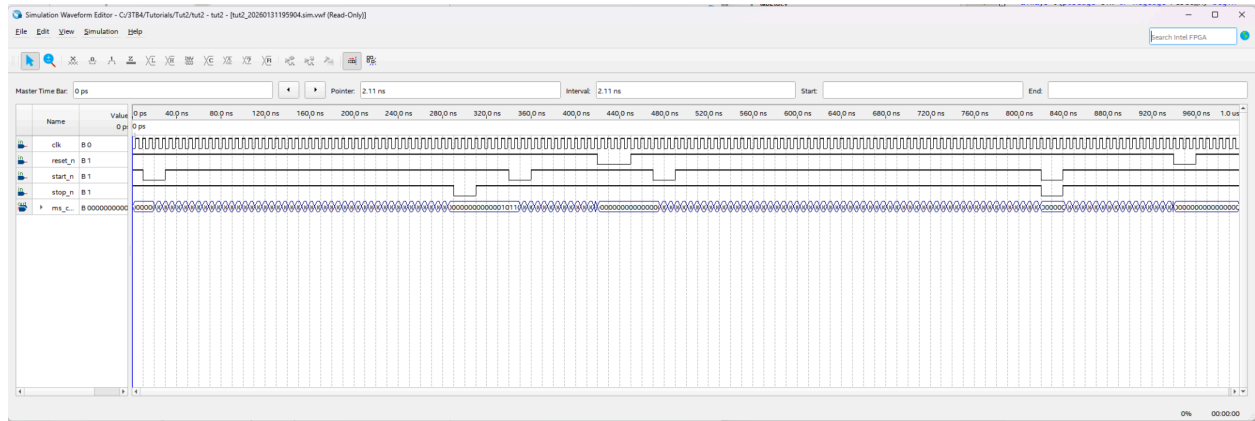


Figure 9b. Screenshot of the simulation of the Counter.

Part 4: Top-Level Module

The top-level module integrates all the connected sub-modules, such as the clock divider, counter, binary-to-BCD converter, and the seven-segment decoder, into a functional stopwatch system. The 50 MHz clock is first reduced to a 1 kHz signal using the clock divider, which slows it down to milliseconds. This reduced clock drives the counter module and increments in milliseconds when enabled. The counter output is converted from binary to BCD format using the hex_to_bcd_converter module so that it can correctly display the six seven-segment displays. The push buttons (KEY inputs) are reset (KEY[0]) start (KEY[1]), and stop/resume (KEY[2]). The top module acts as the interface between the FPGA hardware and the logical design.

Flow Summary	
 <<Filter>>	
Flow Status	Successful - Thu Feb 12 00:05:07 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	tut2
Top-level Entity Name	lab2tut
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	154 / 32,070 (< 1 %)
Total registers	65
Total pins	46 / 457 (10 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 10. Compilation report for Top Level Module.

Table 4. Summary for Components for Top Level Module in Lab 2 Tutorial Part 2.

Feature	Quantity
Total number of logic elements	154
Total number of registers	65
Total number of pins	46
Max number of logic elements that can fit on FPGA	32,070

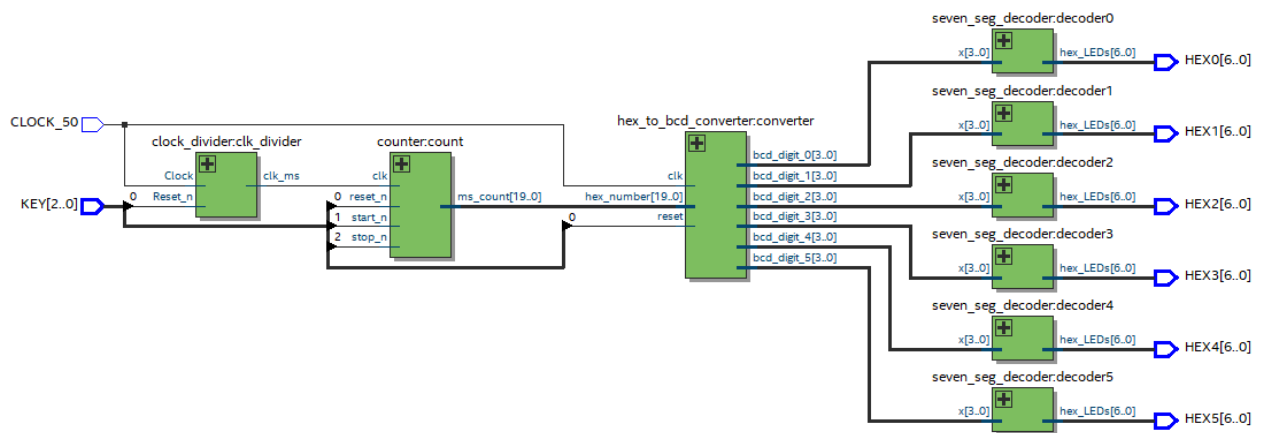


Figure 11. Screenshot of the implemented Top Level Module.

```

1 // TOP LEVEL MODULE
2
3 module lab2tut (input CLOCK_50,
4                 input [2:0] KEY, // KEY[0] = reset, KEY[1] = start, KEY[2] = stop
5                 output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5);
6
7     wire clk_en;
8     wire [19:0] ms;
9     wire [3:0] digit0,digit1,digit2,digit3,digit4,digit5;
10
11     clock_divider #(C.factor(50000)) clk_divider(.clock(CLOCK_50),
12                                                  .Reset_n(KEY[0]),
13                                                  .clk_ms(clk_en));
14
15     counter count(.clk(clk_en),
16                  .reset_n(KEY[0]),
17                  .start_n(KEY[1]),
18                  .stop_n(KEY[2]),
19                  .ms_count(ms));
20
21     hex_to_bcd_converter converter(.clk(CLOCK_50),
22                                   .reset(KEY[0]),
23                                   .hex_number(ms),
24                                   .bcd_digit_0(digit0),
25                                   .bcd_digit_1(digit1),
26                                   .bcd_digit_2(digit2),
27                                   .bcd_digit_3(digit3),
28                                   .bcd_digit_4(digit4),
29                                   .bcd_digit_5(digit5));
30
31     seven_seg_decoder decoder0(.x(digit0), .hex_LEDs(HEX0));
32     seven_seg_decoder decoder1(.x(digit1), .hex_LEDs(HEX1));
33     seven_seg_decoder decoder2(.x(digit2), .hex_LEDs(HEX2));
34     seven_seg_decoder decoder3(.x(digit3), .hex_LEDs(HEX3));
35     seven_seg_decoder decoder4(.x(digit4), .hex_LEDs(HEX4));
36     seven_seg_decoder decoder5(.x(digit5), .hex_LEDs(HEX5));
37
38 endmodule
39
40 // Seven segment display module
41 module seven_seg_decoder(input [3:0] x, output reg [6:0] hex_LEDs);
42     always @(*) begin
43         case (x)
44             4'b0000: hex_LEDs[6:0] = 7'b1000000; // decimal 0
45             4'b0001: hex_LEDs[6:0] = 7'b1111001; // decimal 1
46             4'b0010: hex_LEDs[6:0] = 7'b0100100; // decimal 2
47             4'b0011: hex_LEDs[6:0] = 7'b0110000; // decimal 3
48             4'b0100: hex_LEDs[6:0] = 7'b0011001; // decimal 4
49             4'b0101: hex_LEDs[6:0] = 7'b0010010; // decimal 5
50             4'b0110: hex_LEDs[6:0] = 7'b0000010; // decimal 6
51             4'b0111: hex_LEDs[6:0] = 7'b1111000; // decimal 7
52             4'b1000: hex_LEDs[6:0] = 7'b0000000; // decimal 8
53             4'b1001: hex_LEDs[6:0] = 7'b0010000; // decimal 9
54             4'b1111: hex_LEDs[6:0] = 7'b1111111; // OFF
55             default: hex_LEDs[6:0] = 7'b1111111; // default behaviour
56         endcase
57     end
58 endmodule
59

```

Figure 12. Screenshot of the module of the Top Level Module.

	at ▼	From	To	Assignment Name	Value	Enabled	Entity	Comment	Tag
1	✓		in CLOCK_50	Location	PIN_AF14	Yes			
2	✓		out HEX0[0]	Location	PIN_AE26	Yes			
3	✓		out HEX0[1]	Location	PIN_AE27	Yes			
4	✓		out HEX0[2]	Location	PIN_AE28	Yes			
5	✓		out HEX0[3]	Location	PIN_AG27	Yes			
6	✓		out HEX0[4]	Location	PIN_AF28	Yes			
7	✓		out HEX0[5]	Location	PIN_AG28	Yes			
8	✓		out HEX0[6]	Location	PIN_AH28	Yes			
9	✓		out HEX1[0]	Location	PIN_AJ29	Yes			
10	✓		out HEX1[1]	Location	PIN_AH29	Yes			
11	✓		out HEX1[2]	Location	PIN_AH30	Yes			
12	✓		out HEX1[3]	Location	PIN_AG30	Yes			
13	✓		out HEX1[4]	Location	PIN_AF29	Yes			
14	✓		out HEX1[5]	Location	PIN_AF30	Yes			
15	✓		out HEX1[6]	Location	PIN_AD27	Yes			
16	✓		out HEX2[0]	Location	PIN_AB23	Yes			
17	✓		out HEX2[1]	Location	PIN_AE29	Yes			
18	✓		out HEX2[2]	Location	PIN_AD29	Yes			
19	✓		out HEX2[3]	Location	PIN_AC28	Yes			
20	✓		out HEX2[4]	Location	PIN_AD30	Yes			
21	✓		out HEX2[5]	Location	PIN_AC29	Yes			
22	✓		out HEX2[6]	Location	PIN_AC30	Yes			
23	✓		out HEX3[0]	Location	PIN_AD26	Yes			
24	✓		out HEX3[1]	Location	PIN_AC27	Yes			
25	✓		out HEX3[2]	Location	PIN_AD25	Yes			
26	✓		out HEX3[3]	Location	PIN_AC25	Yes			
27	✓		out HEX3[4]	Location	PIN_AB28	Yes			
28	✓		out HEX3[5]	Location	PIN_AB25	Yes			
29	✓		out HEX3[6]	Location	PIN_AB22	Yes			
30	✓		out HEX4[0]	Location	PIN_AA24	Yes			
31	✓		out HEX4[1]	Location	PIN_Y23	Yes			
32	✓		out HEX4[2]	Location	PIN_Y24	Yes			
33	✓		out HEX4[3]	Location	PIN_W22	Yes			
34	✓		out HEX4[4]	Location	PIN_W24	Yes			
35	✓		out HEX4[5]	Location	PIN_V23	Yes			
36	✓		out HEX4[6]	Location	PIN_W25	Yes			
37	✓		out HEX5[0]	Location	PIN_V25	Yes			
38	✓		out HEX5[1]	Location	PIN_AA28	Yes			
39	✓		out HEX5[2]	Location	PIN_Y27	Yes			
40	✓		out HEX5[3]	Location	PIN_AB27	Yes			
41	✓		out HEX5[4]	Location	PIN_AB26	Yes			
42	✓		out HEX5[5]	Location	PIN_AA26	Yes			
43	✓		out HEX5[6]	Location	PIN_AA25	Yes			
44	✓		in KEY[0]	Location	PIN_AA14	Yes			
45	✓		in KEY[1]	Location	PIN_AA15	Yes			
46	✓		in KEY[2]	Location	PIN_W15	Yes			

Figure 13. Screenshot of the pins used for the Top Level Module.

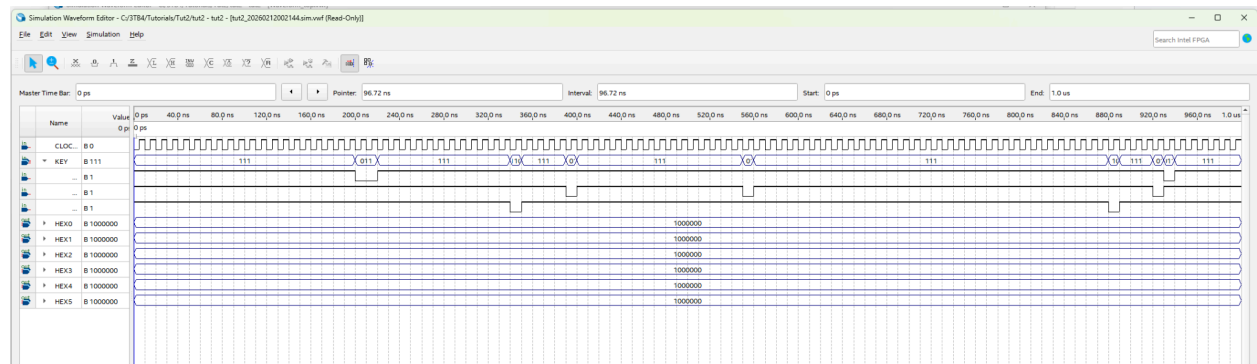


Figure 14. Screenshot of the simulation of the Top Level Module.

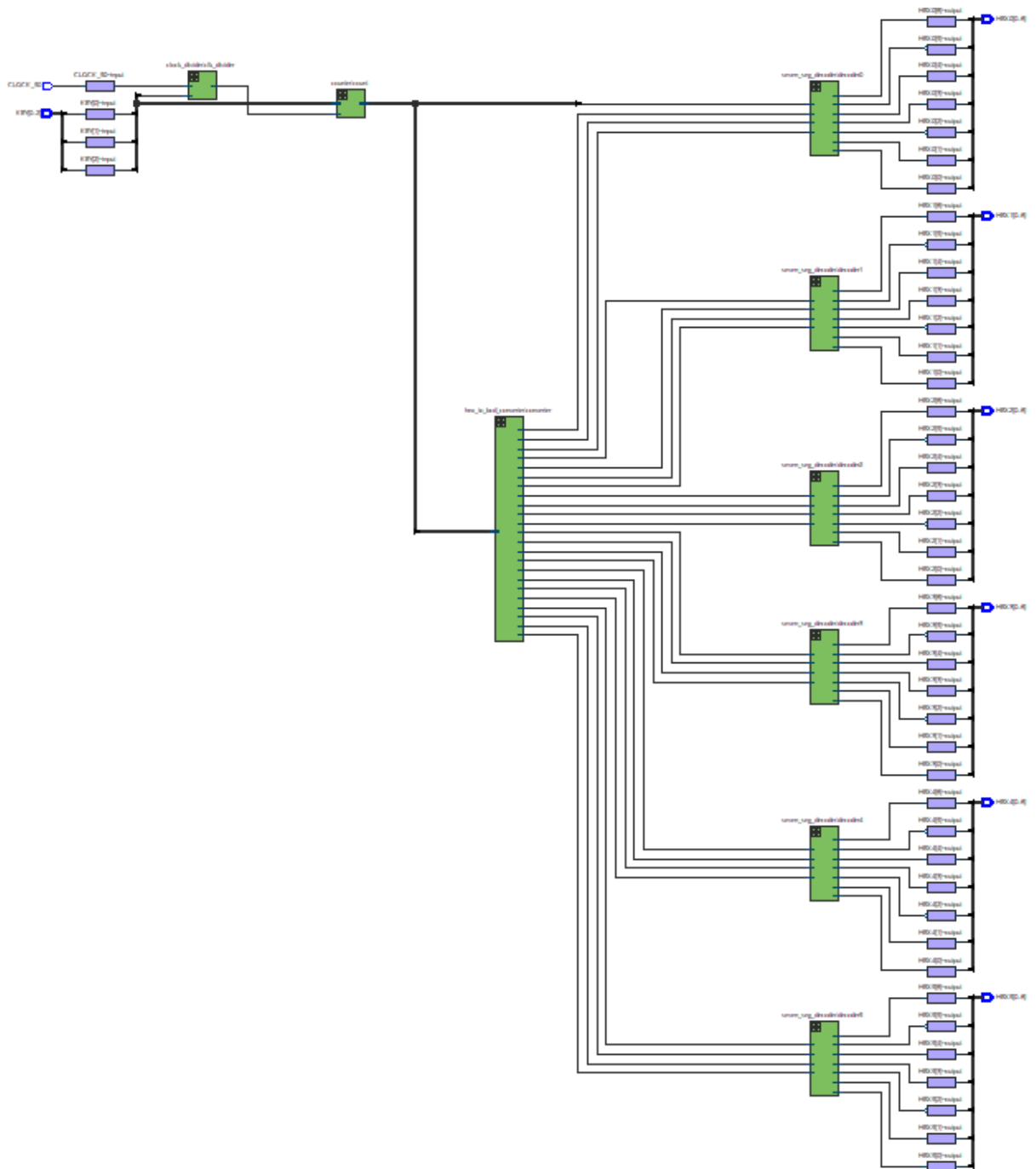


Figure 15. Post-Mapping image.

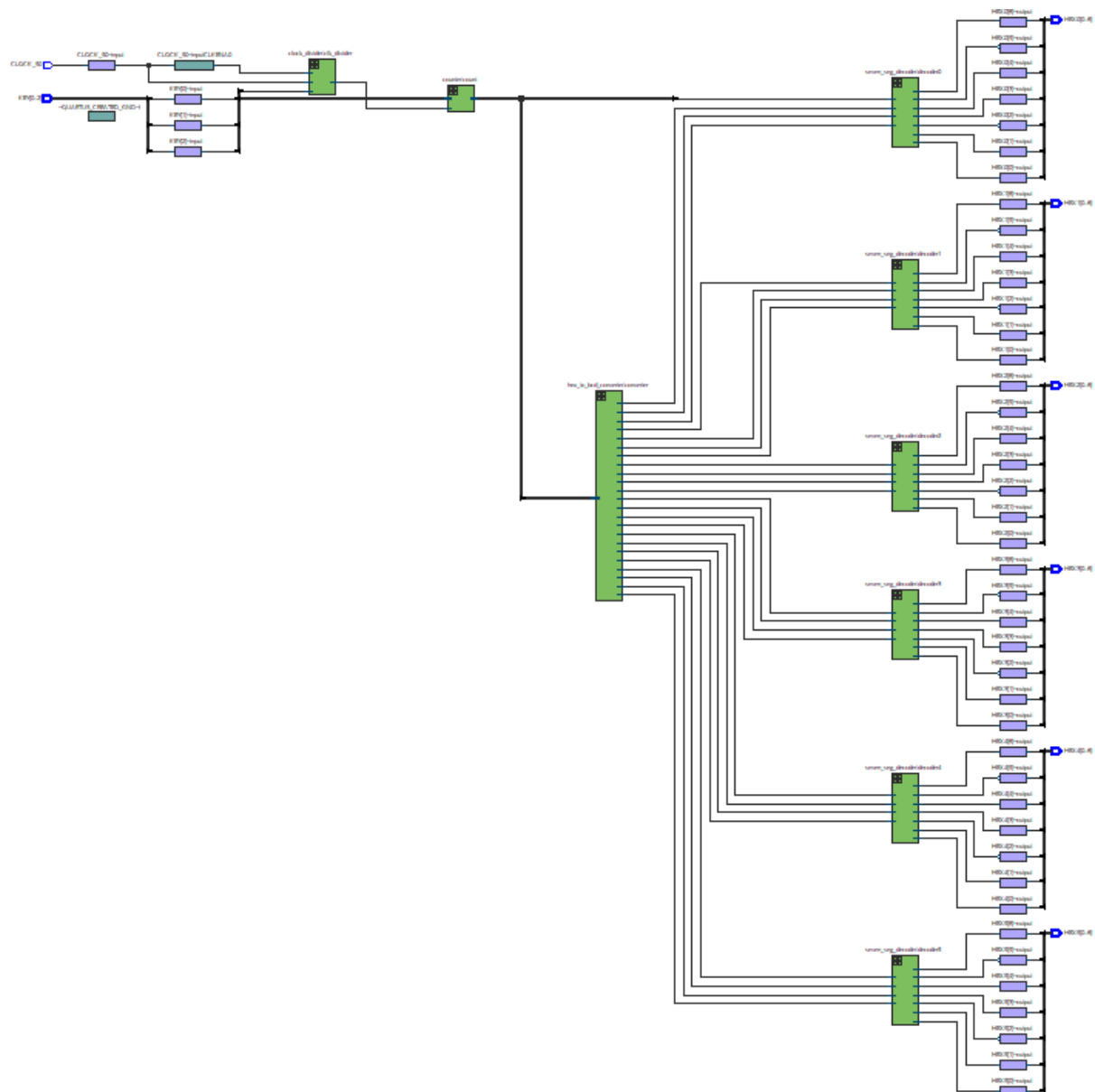


Figure 16. Post-Fitting image.

Top View - Wire Bond Cyclone V - 5CSEMA5F31C6

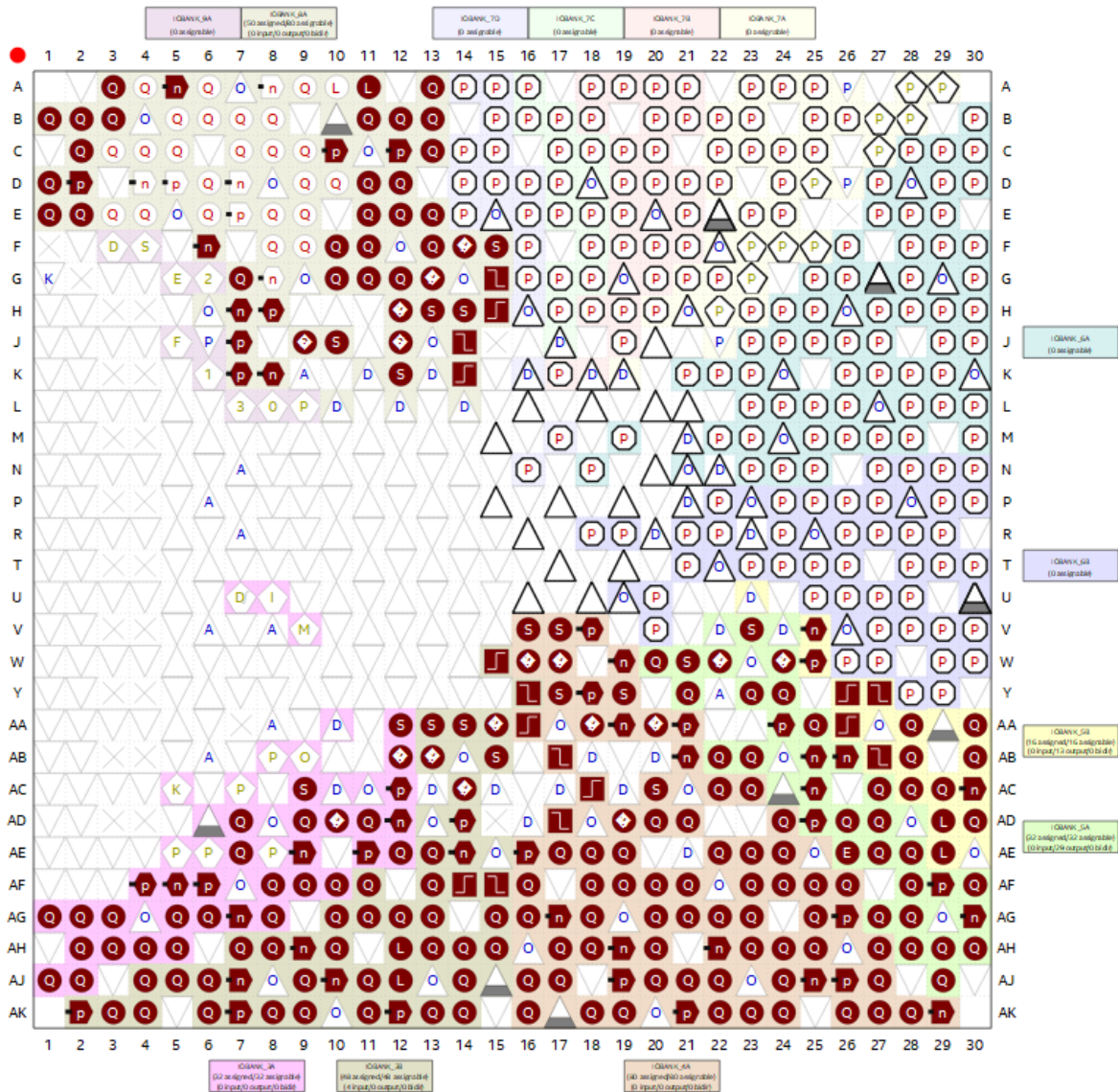


Figure 17. Pin planner image.

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	
CLOCK_50	Input	PIN_AF14	3B	B3B_NO	PIN_AF14	2.5 V		12mA (default)	
HEX0[6]	Output	PIN_AH28	5A	B5A_NO	PIN_AH28	2.5 V		12mA (default)	1 (defa
HEX0[5]	Output	PIN_AG28	5A	B5A_NO	PIN_AG28	2.5 V		12mA (default)	1 (defa
HEX0[4]	Output	PIN_AF28	5A	B5A_NO	PIN_AF28	2.5 V		12mA (default)	1 (defa
HEX0[3]	Output	PIN_AG27	5A	B5A_NO	PIN_AG27	2.5 V		12mA (default)	1 (defa
HEX0[2]	Output	PIN_AE28	5A	B5A_NO	PIN_AE28	2.5 V		12mA (default)	1 (defa
HEX0[1]	Output	PIN_AE27	5A	B5A_NO	PIN_AE27	2.5 V		12mA (default)	1 (defa
HEX0[0]	Output	PIN_AE26	5A	B5A_NO	PIN_AE26	2.5 V		12mA (default)	1 (defa
HEX1[6]	Output	PIN_AD27	5A	B5A_NO	PIN_AD27	2.5 V		12mA (default)	1 (defa
HEX1[5]	Output	PIN_AF30	5A	B5A_NO	PIN_AF30	2.5 V		12mA (default)	1 (defa
HEX1[4]	Output	PIN_AF29	5A	B5A_NO	PIN_AF29	2.5 V		12mA (default)	1 (defa
HEX1[3]	Output	PIN_AG30	5A	B5A_NO	PIN_AG30	2.5 V		12mA (default)	1 (defa
HEX1[2]	Output	PIN_AH30	5A	B5A_NO	PIN_AH30	2.5 V		12mA (default)	1 (defa
HEX1[1]	Output	PIN_AH29	5A	B5A_NO	PIN_AH29	2.5 V		12mA (default)	1 (defa
HEX1[0]	Output	PIN_AJ29	5A	B5A_NO	PIN_AJ29	2.5 V		12mA (default)	1 (defa
HEX2[6]	Output	PIN_AC30	5B	B5B_NO	PIN_AC30	2.5 V		12mA (default)	1 (defa
HEX2[5]	Output	PIN_AC29	5B	B5B_NO	PIN_AC29	2.5 V		12mA (default)	1 (defa
HEX2[4]	Output	PIN_AD30	5B	B5B_NO	PIN_AD30	2.5 V		12mA (default)	1 (defa
HEX2[3]	Output	PIN_AC28	5B	B5B_NO	PIN_AC28	2.5 V		12mA (default)	1 (defa
HEX2[2]	Output	PIN_AD29	5B	B5B_NO	PIN_AD29	2.5 V		12mA (default)	1 (defa
HEX2[1]	Output	PIN_AE29	5B	B5B_NO	PIN_AE29	2.5 V		12mA (default)	1 (defa
HEX2[0]	Output	PIN_AB23	5A	B5A_NO	PIN_AB23	2.5 V		12mA (default)	1 (defa
HEX3[6]	Output	PIN_AB22	5A	B5A_NO	PIN_AB22	2.5 V		12mA (default)	1 (defa
HEX3[5]	Output	PIN_AB25	5A	B5A_NO	PIN_AB25	2.5 V		12mA (default)	1 (defa
HEX3[4]	Output	PIN_AB28	5B	B5B_NO	PIN_AB28	2.5 V		12mA (default)	1 (defa
HEX3[3]	Output	PIN_AC25	5A	B5A_NO	PIN_AC25	2.5 V		12mA (default)	1 (defa
HEX3[2]	Output	PIN_AD25	5A	B5A_NO	PIN_AD25	2.5 V		12mA (default)	1 (defa
HEX3[1]	Output	PIN_AC27	5A	B5A_NO	PIN_AC27	2.5 V		12mA (default)	1 (defa
HEX3[0]	Output	PIN_AD26	5A	B5A_NO	PIN_AD26	2.5 V		12mA (default)	1 (defa
HEX4[6]	Output	PIN_W25	5B	B5B_NO	PIN_W25	2.5 V		12mA (default)	1 (defa
HEX4[5]	Output	PIN_V23	5A	B5A_NO	PIN_V23	2.5 V		12mA (default)	1 (defa
HEX4[4]	Output	PIN_W24	5A	B5A_NO	PIN_W24	2.5 V		12mA (default)	1 (defa
HEX4[3]	Output	PIN_W22	5A	B5A_NO	PIN_W22	2.5 V		12mA (default)	1 (defa
HEX4[2]	Output	PIN_Y24	5A	B5A_NO	PIN_Y24	2.5 V		12mA (default)	1 (defa
HEX4[1]	Output	PIN_Y23	5A	B5A_NO	PIN_Y23	2.5 V		12mA (default)	1 (defa
HEX4[0]	Output	PIN_AA24	5A	B5A_NO	PIN_AA24	2.5 V		12mA (default)	1 (defa
HEX5[6]	Output	PIN_AA25	5A	B5A_NO	PIN_AA25	2.5 V		12mA (default)	1 (defa
HEX5[5]	Output	PIN_AA26	5B	B5B_NO	PIN_AA26	2.5 V		12mA (default)	1 (defa
HEX5[4]	Output	PIN_AB26	5A	B5A_NO	PIN_AB26	2.5 V		12mA (default)	1 (defa
HEX5[3]	Output	PIN_AB27	5B	B5B_NO	PIN_AB27	2.5 V		12mA (default)	1 (defa
HEX5[2]	Output	PIN_Y27	5B	B5B_NO	PIN_Y27	2.5 V		12mA (default)	1 (defa
HEX5[1]	Output	PIN_AA28	5B	B5B_NO	PIN_AA28	2.5 V		12mA (default)	1 (defa
HEX5[0]	Output	PIN_V25	5B	B5B_NO	PIN_V25	2.5 V		12mA (default)	1 (defa
KEY[2]	Input	PIN_W15	3B	B3B_NO	PIN_W15	2.5 V		12mA (default)	
KEY[1]	Input	PIN_AA15	3B	B3B_NO	PIN_AA15	2.5 V		12mA (default)	
KEY[0]	Input	PIN_AA14	3B	B3B_NO	PIN_AA14	2.5 V		12mA (default)	

Figure 18. Pin planner table.

Lab 2 Prelab Report

Overview

The goal of this pre-lab was to design and implement a device for a reaction-time contest. This was achieved by designing an LFSR-based random number generator to generate variable delays, designing a Finite State Machine (FSM) to control the overall system, and writing Verilog code for the FSM and other necessary modules. Other modules, such as those for blinking the HEX displays, were also developed as needed.

Part 1: LFSR Counter for Random Number Generation

The Galois Linear Feedback Shift Register (LFSR) module was designed to generate pseudo-random numbers used to create unpredictable delays for a reaction-time contest between 2-players. This updates once per clock cycle by shifting the register contents and inserting a feedback bit computed from the specific tap positions 14, 5, 3, and 1 by using XOR operations. In the Galois configuration, the bits that are not tapped will shift one position to the right unchanged, whereas the taps are XORed with the output bit and stored in another position. The Galois is chosen for its efficiency since only the output bit needs to be checked. If the output bit is 1, the tapped bits are inverted using XOR operations. If the output bit is 0, it remains unchanged.

Since the LFSR can become stuck in a repeated state, the register was initialized to a non-locking seed value of 14'b11111111111111 that prevents the invalid lock-up states for XOR. The module continuously updates the values until the generated values fall in the range of 1000 to 5000. Once a valid value within the range is generated, the module stores the values into an output register called random and asserts a rnd_ready signal. The LFSR is disabled by disabling enable and prevents further updates until a resume signal is received. This ensures that the delay remains constant throughout the reaction phase and does not change during gameplay.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Thu Feb 12 00:26:53 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab2_rng
Top-level Entity Name	random
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	15 / 32,070 (< 1 %)
Total registers	30
Total pins	18 / 457 (4 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 19. Screenshot of the LFSR Random compilation report.

Table 5. Summary for Components for the Random Module in PreLab 2.

Feature	Quantity
Total number of logic elements	15
Total number of registers	30
Total number of pins	18
Max number of logic elements that can fit on FPGA	32,070

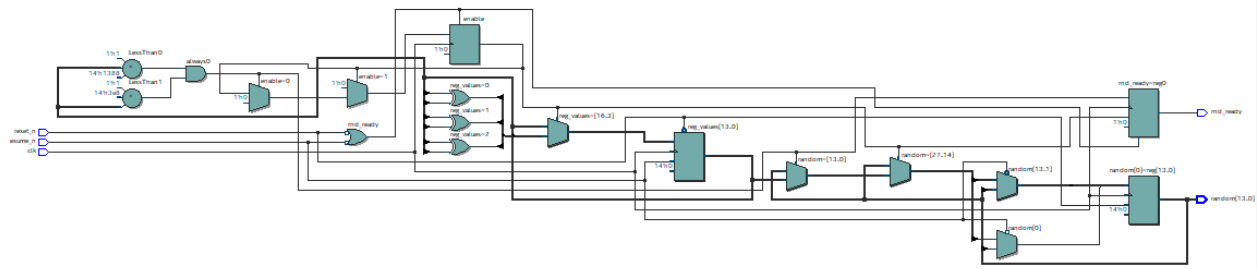


Figure 20. Screenshot of the implemented Random Module.

```

1  module random (input clk, reset_n, resume_n,
2                  output reg [13:0] random,
3                  output reg rnd_ready);
4      //for 14 bits Linear Feedback Shift Register,
5      //the Taps that need to be XORed or XORed are: 14, 5, 3, 1
6      wire xnor_taps, and_allbits, feedback;
7      reg [13:0] reg_values;
8      reg enable = 1;
9
10     always @ (posedge clk, negedge reset_n, negedge resume_n) begin
11         if (!reset_n) begin
12             reg_values <= 14'b11111111111111;
13             //the LFSR cannot be all 0 at beginning.
14             enable <= 1;
15             rnd_ready <= 0;
16         end
17
18         else if (!resume_n) begin
19             enable <= 1;
20             rnd_ready <= 0;
21             reg_values <= reg_values;
22         end
23
24         else begin
25             if (enable) begin
26                 reg_values[13] <= reg_values[0];
27                 reg_values[12:5] <= reg_values[13:6];
28                 reg_values[4] <= reg_values[0] ^ reg_values[5];
29                 // tap 5 of the diagram from the lab manual
30                 reg_values[3] <= reg_values[4];
31                 reg_values[2] <= reg_values[0] ^ reg_values[3];
32                 // tap 3 of the diagram from the lab manual
33                 reg_values[1] <= reg_values[2];
34                 reg_values[0] <= reg_values[0] ^ reg_values[1];
35                 // tap 1 of the diagram from the lab manual
36
37                 // confirm sure the random number is between 1000 and 5000
38                 if (reg_values <= 5000 && reg_values >= 1000) begin
39                     rnd_ready <= 1'b1;
40                     enable <= 1'b0;
41                     random <= reg_values;
42                 end
43                 else begin
44                     rnd_ready <= 1'b0;
45                 end
46             end //end of ENABLE.
47         end
48     end
49 endmodule

```

Figure 21. Screenshot of the module of the Random Module.

Part 2: FSM

The Finite State Machine (FSM) was implemented to coordinate the overall behaviour of the reaction-time contest system based on the different rules of the game. The FSM controls state transitions between the nine different states, as shown in Figure 22, such as Reset, Start, Wait, Counting, and so forth. Each state defines the system's behaviours, including the seven-segment display blinking, delay timing, score update, reaction timing, and display selection.

The FSM transitions are determined by push-button inputs, timing thresholds, and the randomly generated delay from the LFSR module. It starts in the RESET state (State 1) either when `reset=0`, or during the start of the game. It transitions into the START state (State 2), and if `reset=1`, the LEDs start blinking to indicate a new round is starting. Once 5000 milliseconds in the counter elapsed, the FSM transitions to the WAIT state (State 3), where the LEDs remain off until the random delay generated by the LFSR is finished. After the random delay, the FSM enters the COUNTING state (State 4), enabling the reaction timer and monitoring the player's inputs. If a player presses the button before the reaction phase begins, the FSM transitions to the CHEATING state corresponding to if Player 1, Player 2, or both cheated, State 5 to 7, respectively. If a valid reaction is detected within the reaction phase, the FSM transitions to the appropriate winning state, State 8 if Player 1 wins, and State 9 if Player 2 wins and updates the scores to the respective players. The state-driven design ensures an organized control flow and predictable behaviours based on states and transition conditions while incorporating the LFSR for randomized timing to ensure fairness in the reaction-based game.

INPUTS		OUTPUTS	
• P1		• BLINK	
• P2		• DISP_COUNTER	
• RESUME		• HEX1	
• RESET		• HEX2	
• COUNTER		• HEX8	
• RAND_NUM		• P1_WIN	
		• P2_WIN	
STATES			
• State 1 =>	Reset		
• State 2 =>	Start		
• State 3 =>	Wait		
• State 4 =>	Counting		
• State 5 =>	Player 1 cheat		
• State 6 =>	Player 2 cheat		
• State 7 =>	Player 1 and 2 cheat		
• State 8 =>	Player 1 win		
• State 9 =>	Player 2 win		

Figure 22. Screenshot of the inputs, outputs, and states for the FSM.

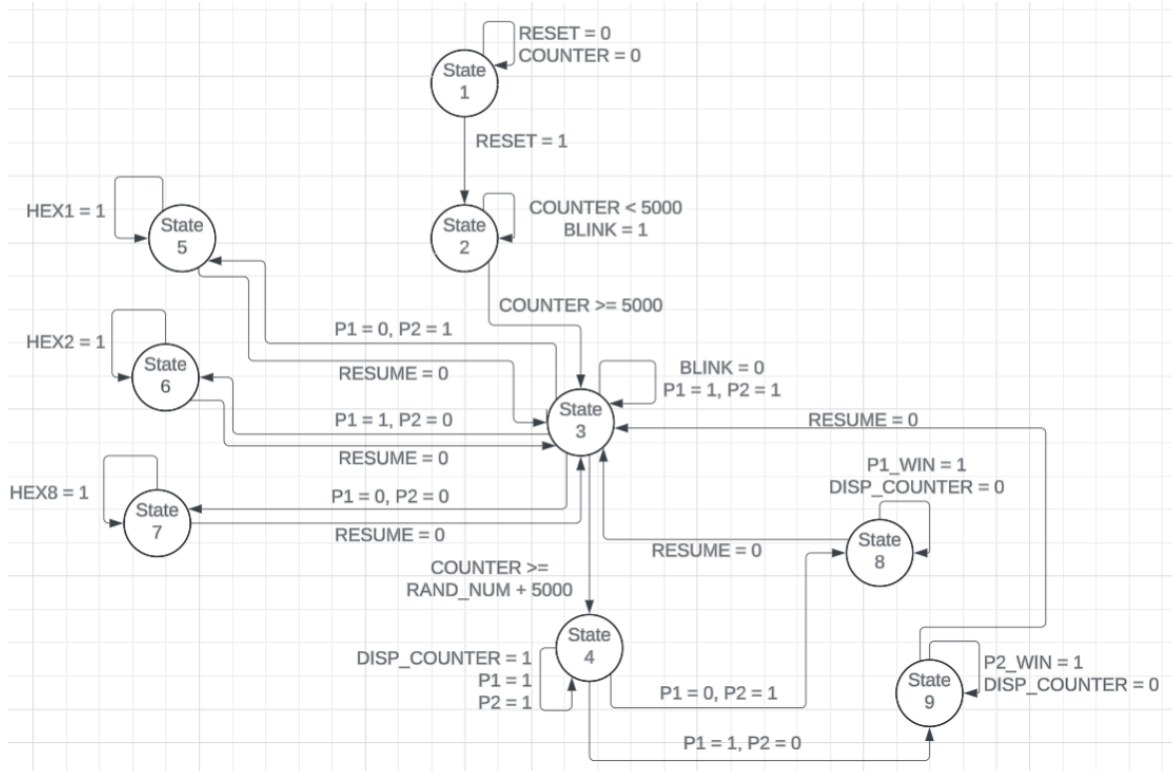


Figure 23a. Screenshot of the FSM without RESET explicitly drawn from each state.

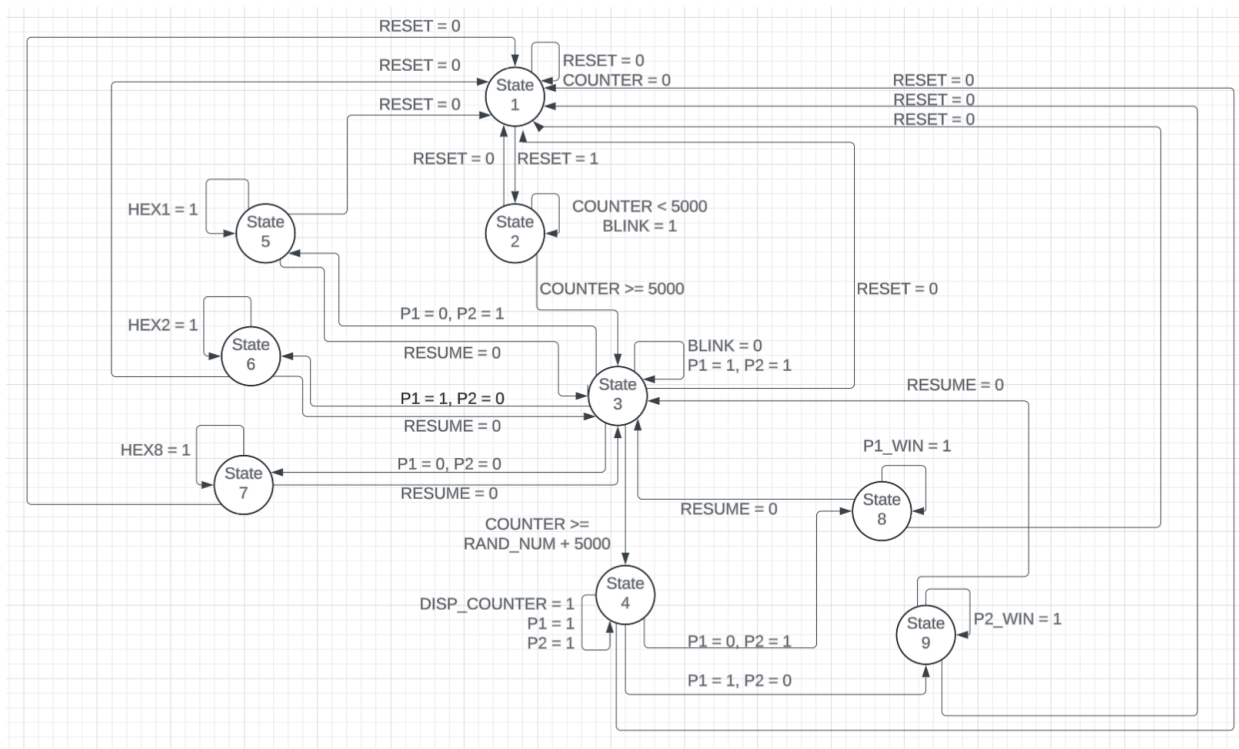


Figure 23b. Screenshot of the FSM with RESET explicitly drawn from each state.

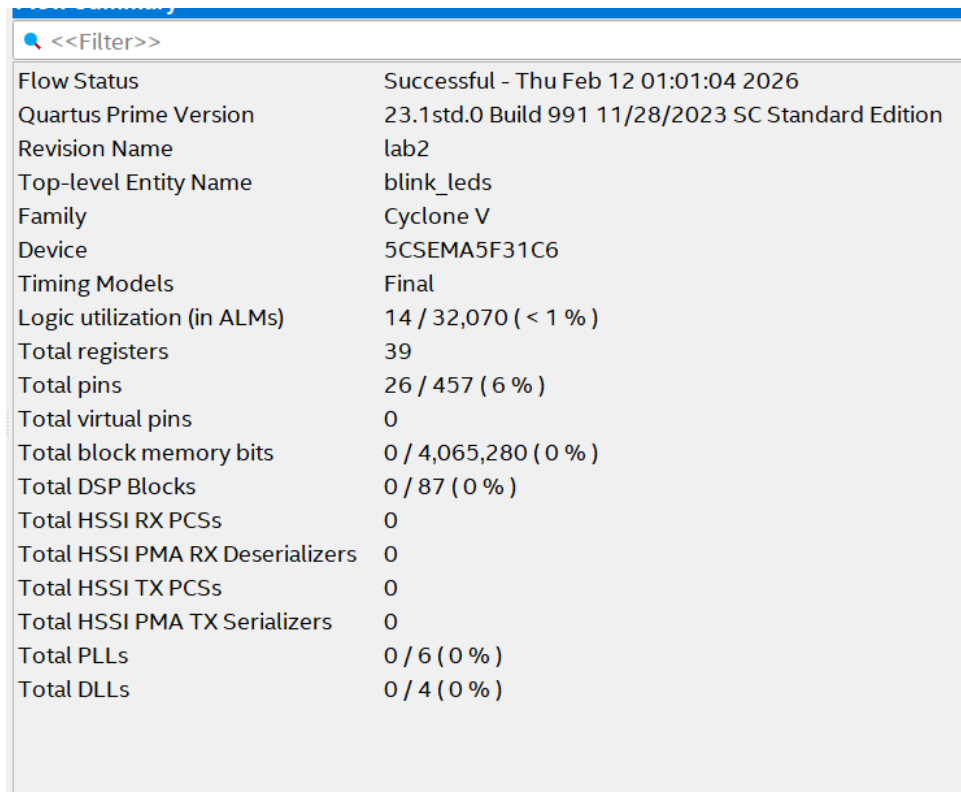
The select line for the multiplexer (mux) has 4 possibilities for the seven-segment display, including 00 (off), 01 (BCD_out), 10 (blink), and 11 (cheat_disp). Some features of the FSM were designed based on active low, such as the push-buttons (KEYS), where “1” indicates off (not pressed) and “0” means on (pressed). If the display counter was less than 80, it would result in “CHEAT” since that is considered faster than human reaction time. If KEY[1] (reset) is pressed, the FSM can be reset at any state, but this was not drawn on the FSM in Figure 23a to keep it organized and not cluttered. However, Figure 23b displays what happens when KEY[1] (reset) is pressed at each state. In addition, the states and the statuses of the corresponding signals at each state are outlined in Table 6 and were not included on the FSM diagram directly due to a lack of space and organizational reasons.

Table 6. States and corresponding inputs.

State	State Name	Input
1	RESET	select = 00 disp_counter_enable = 0
2	START	select = 10 (blink) main_counter_start = 0 disp_counter_reset = 0
3	WAIT	select = 00 disp_counter_reset = 0
4	COUNTING	select = 01 (display disp_counter) disp_counter_reset = 1 disp_counter_enable = 1 disp_counter_start = 0
5	P1_CHEAT	select = 11 (cheat) disp_counter_enable = 0 cheat_disp = 24'h111111
6	P2_CHEAT	select = 11 (cheat) disp_counter_enable = 0 cheat_disp = 24'h222222
7	BOTH_CHEAT	select = 11 (cheat) disp_counter_enable = 0 cheat_disp = 24'h888888
8	P1_WIN	select = 01 (display disp_counter, winning time) disp_counter_enable = 0 disp_counter_start = 1 P1_score = P1_score + 1 (LED lights up)
9	P2_WIN	select = 01 (display disp_counter, winning time) disp_counter_enable = 0 disp_counter_start = 1 P2_score = P2_score + 1 (LED lights up)
Default	Default Conditions	select = 00 disp_counter_enable = 0

Part 3: FSM Blink LEDs

The FSM blink LEDs module was implemented to generate the blinking display used during the FSM preparation phase (START state). The module uses a clock-driven counter and a “factor” parameter to alternate the 24-bit LED output between ON (all 1) or OFF (all 0) patterns to produce a visible blinking effect. The blinking output is not continuously displayed by default; it is selected based on the FSM state using the multiplexer display. When the FSM enters the START state, it sets the mux select signal to choose the blink module output “sel = 2'b10”. This allows the HEX display LEDs to blink on and off as a “get ready” cue. When the FSM transitions to the WAIT or COUNTING state, the signal changes so that the blink output is no longer routed to the display, ensuring the blink only occurs during the START state.



<<Filter>>	
Flow Status	Successful - Thu Feb 12 01:01:04 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab2
Top-level Entity Name	blink_leds
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	14 / 32,070 (< 1 %)
Total registers	39
Total pins	26 / 457 (6 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 24. Screenshot of the HEX LEDs Blinking module compilation report.

Table 7. Summary for Components for the HEX LEDs Blinking Module in PreLab 2.

Feature	Quantity
Total number of logic elements	14
Total number of registers	39
Total number of pins	26
Max number of logic elements that can fit on FPGA	32,070

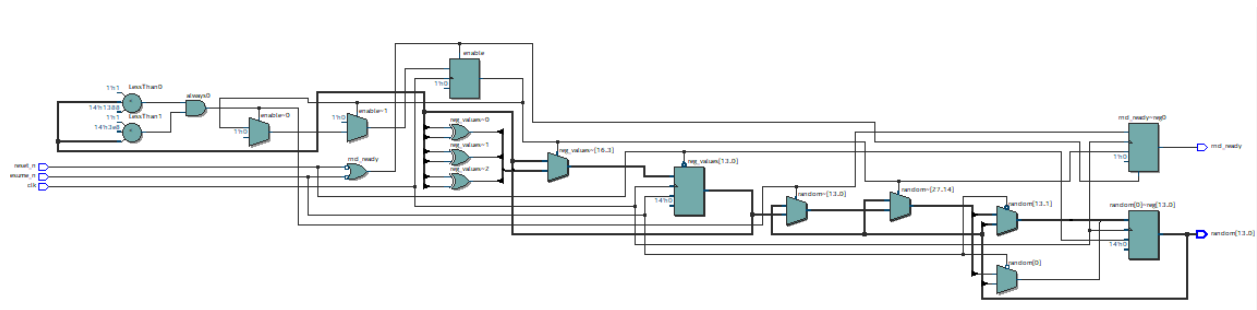


Figure 25. Screenshot of the implemented HEX LEDs Blinking Module.

Code

Lab Tutorial 2

Binary to BCD Converter

// CONVERTS HEX TO BCD

```
module hex_to_bcd_converter(input wire clk, reset,
                           input wire [19:0]
                           hex_number,
                           output [3:0]
                           bcd_digit_0,bcd_digit_1,bcd_digit_2,bcd_digit_3, bcd_digit_4, bcd_digit_5);
    // DE1-SoC has 6 7_seg_LEDs, 20 bits can represent decimal 999999.
    //This module is designed to convert a 20-bit binary representation to BCD
    integer i, k, j;
    wire [19:0] hex_number1; // the last 20 bits of input hex_number
    reg [3:0] bcd_digit [5:0]; // 6 BCD for 0-5
    assign hex_number1 = hex_number[19:0];

    assign bcd_digit_0 = bcd_digit[0];
    assign bcd_digit_1 = bcd_digit[1];
    assign bcd_digit_2 = bcd_digit[2];
    assign bcd_digit_3 = bcd_digit[3];
    assign bcd_digit_4 = bcd_digit[4];
    assign bcd_digit_5 = bcd_digit[5];

    always @ (*) begin
        if (!reset) begin
            //set all 6 digits to 0
            for (j=5 ; j>=0 ; j=j-1) begin
                bcd_digit[j] = 4'b0000;
            end
        end
        //set all 6 digits to 0
        for (j=5 ; j>=0 ; j=j-1) begin
            bcd_digit[j] = 4'b0000;
        end

        //shift 20 times
        for (i=19; i>=0; i=i-1)
            begin

                //check all 6 BCD tetrads
```

```

for (k=5; k>=0; k=k-1)
begin
    // if >=5 then add 3
    if (bcd_digit[k] >= 5) begin
        bcd_digit[k] = bcd_digit[k] + 4'b0011;
    end
end

//shift one bit of BIN/HEX left
//for the first 5 tetrads
for (k=5; k>=1; k=k-1)
begin
    bcd_digit[k]=bcd_digit[k] << 1;
    bcd_digit[k][0]=bcd_digit[k-1][3];
end

//shift one bit of BIN/HEX left, for the last tetrad
/* fill your code here */
bcd_digit[0] = bcd_digit[0] << 1;
bcd_digit[0][0] = hex_number[i];

    end //end for loop
end //end of always.
endmodule

```

Clock Divider

// CLOCK DIVIDER => MAKES FAST CLOCK (50 MHz) --> SLOWER (1 KHz)

```
module clock_divider (input Clock, Reset_n, output reg clk_ms);
    parameter factor = 50000; //50000; // 32'h000061a7;

    reg [31:0] countQ;

    always @ (posedge Clock, negedge Reset_n) begin
        if (!Reset_n) begin
            countQ <= 32'b0; // reset countQ to 0
            clk_ms <= 1'b0;
        end
        else begin
            if (countQ < factor/2) begin // clk is low
                countQ <= countQ + 1'b1; // increment countQ
                clk_ms <= 1'b1;
            end

            else if (countQ < factor) begin // clk is high
                countQ <= countQ + 1'b1;
                clk_ms <= 1'b0;
            end

            else begin //countQ == factor => nxt clk cycle
                countQ <= 32'b0; // reset countQ
            end
        end
    end
endmodule
```


Counter

// COUNTER

```
module counter(input clk, input reset_n, start_n, stop_n, output reg [19:0] ms_count);
    reg isCounting; // counter flag

    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            ms_count <= 20'b0; // reset counter
            isCounting <= 1'b0; // reset flag
        end

        else begin
            if (!start_n) begin
                isCounting <= 1'b1; // start counting
            end

            else if (!stop_n) begin
                ms_count <= ms_count; // store prev value
                isCounting <= 1'b0; // stop counting
            end

            if (isCounting && stop_n) begin
                ms_count <= ms_count + 1'b1;
            end

        end

    end

end

endmodule
```

Top Level Module

```
// TOP LEVEL MODULE
```

```
module lab2tut (input CLOCK_50,
                input [2:0] KEY, // KEY[0] = reset, KEY[1] = start,
                KEY[2] = stop
                output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4,
                HEX5);
    wire clk_en;
    wire [19:0] ms;
    wire [3:0] digit0,digit1,digit2,digit3,digit4,digit5;

    clock_divider #(.factor(50000)) clk_divider(.Clock(CLOCK_50),

        .Reset_n(KEY[0]),

        .clk_ms(clk_en));

    counter count(.clk(clk_en),
        .reset_n(KEY[0]),
        .start_n(KEY[1]),
        .stop_n(KEY[2]),
        .ms_count(ms));

    hex_to_bcd_converter converter(.clk(CLOCK_50),
        .reset(KEY[0]),
        .hex_number(ms),
        .bcd_digit_0(digit0),
        .bcd_digit_1(digit1),
        .bcd_digit_2(digit2),
        .bcd_digit_3(digit3),
        .bcd_digit_4(digit4),
        .bcd_digit_5(digit5));

    seven_seg_decoder decoder0(.x(digit0), .hex_LEDs(HEX0));
    seven_seg_decoder decoder1(.x(digit1), .hex_LEDs(HEX1));
    seven_seg_decoder decoder2(.x(digit2), .hex_LEDs(HEX2));
    seven_seg_decoder decoder3(.x(digit3), .hex_LEDs(HEX3));
    seven_seg_decoder decoder4(.x(digit4), .hex_LEDs(HEX4));
    seven_seg_decoder decoder5(.x(digit5), .hex_LEDs(HEX5));

endmodule
```

```

// Seven segment display module
module seven_seg_decoder(input [3:0] x, output reg [6:0] hex_LEDs);
    always @(*) begin
        case (x)
            4'b0000: hex_LEDs[6:0] = 7'b1000000; // decimal 0
            4'b0001: hex_LEDs[6:0] = 7'b1111001; // decimal 1
            4'b0010: hex_LEDs[6:0] = 7'b0100100; // decimal 2
            4'b0011: hex_LEDs[6:0] = 7'b0110000; // decimal 3
            4'b0100: hex_LEDs[6:0] = 7'b0011001; // decimal 4
            4'b0101: hex_LEDs[6:0] = 7'b0010010; // decimal 5
            4'b0110: hex_LEDs[6:0] = 7'b0000010; // decimal 6
            4'b0111: hex_LEDs[6:0] = 7'b1111000; // decimal 7
            4'b1000: hex_LEDs[6:0] = 7'b0000000; // decimal 8
            4'b1001: hex_LEDs[6:0] = 7'b0010000; // decimal 9
            4'b1111: hex_LEDs[6:0] = 7'b1111111; // OFF
            default: hex_LEDs[6:0] = 7'b1111111; // default behaviour
        endcase
    end
endmodule

```

Prelab 2

LFSR Random

```
module random (input clk, reset_n, resume_n,
               output reg [13:0] random,
               output reg rnd_ready);
    //for 14 bits Liner Feedback Shift Register,
    //the Taps that need to be XNORed or XORed are: 14, 5, 3, 1
    wire xnor_taps, and_allbits, feedback;
    reg [13:0] reg_values;
    reg enable = 1;

    always @ (posedge clk, negedge reset_n, negedge resume_n) begin
        if (!reset_n) begin
            reg_values <= 14'b11111111111111;
            //the LFSR cannot be all 0 at the beginning.
            enable <= 1;
            rnd_ready <= 0;
        end

        else if (!resume_n) begin
            enable <= 1;
            rnd_ready <= 0;
            reg_values <= reg_values;
        end

        else begin
            if (enable) begin
                reg_values[13] <= reg_values[0];
                reg_values[12:5] <= reg_values[13:6];
                reg_values[4] <= reg_values[0] ^ reg_values[5];
                // tap 5 of the diagram from the lab manual
                reg_values[3] <= reg_values[4];
                reg_values[2] <= reg_values[0] ^ reg_values[3];
                // tap 3 of the diagram from the lab manual
                reg_values[1] <= reg_values[2];
                reg_values[0] <= reg_values[0] ^ reg_values[1];
                // tap 1 of the diagram from the lab manual

                // confirm sure the random number is between 1000 and 5000
                if (reg_values <= 5000 && reg_values >= 1000) begin
                    rnd_ready <= 1'b1;
                    enable <= 1'b0;
                end
            end
        end
    end
end
```

```
        random <= reg_values;
    end
    else begin
        rnd_ready <= 1'b0;
    end
end //end of ENABLE.
end
end
endmodule
```

Seven-Segment Decoder

```
module seven_seg_decoder(input [3:0] x, output [6:0] hex_LEDs);
    reg [6:0] reg_LEDs;

    assign hex_LEDs[0] = (~x[3] & ~x[2] & ~x[1] & x[0]) |
        (x[2] & ~x[1] & ~x[0]) |
        (x[3] & x[2] & x[1]);
    assign hex_LEDs[1] = (~x[3] & x[2] & ~x[1] & x[0]) |
        (~x[3] & x[2] & x[1] & ~x[0]) |
        (x[3] & x[2] & ~x[1] & ~x[0]) |
        (x[3] & x[1] & x[0]);

    assign hex_LEDs[6:2]=reg_LEDs[6:2];

    always @(*)
    begin
        case (x)
            4'b0000: reg_LEDs[6:2]=5'b10000; //7'b1000000 decimal 0
            4'b0001: reg_LEDs[6:2]=5'b11110; //7'b1111001 decimal 1
            4'b0010: reg_LEDs[6:2]=5'b01001; //7'b0100100 decimal 2
            4'b0011: reg_LEDs[6:2]=5'b01100; //7'b0110000 decimal 3
            4'b0100: reg_LEDs[6:2]=5'b00110; //7'b0011001 decimal 4
            4'b0101: reg_LEDs[6:2]=5'b00100; //7'b0010010 decimal 5
            4'b0110: reg_LEDs[6:2]=5'b00000; //7'b0000010 decimal 6
            4'b0111: reg_LEDs[6:2]=5'b11110; //7'b1111000 decimal 7
            4'b1000: reg_LEDs[6:2]=5'b00000; //7'b0000000 decimal 8
            4'b1001: reg_LEDs[6:2]=5'b00100; //7'b0010000 decimal 9
                        4'b1010: reg_LEDs[6:2]=5'b01000; //7'b0001000 A
                        4'b1011: reg_LEDs[6:2]=5'b00011; //7'b0000011 b
                        4'b1100: reg_LEDs[6:2]=5'b00110; //7'b1000110 C
                        4'b1101: reg_LEDs[6:2]=5'b00001; //7'b0100001 d
                        4'b1110: reg_LEDs[6:2]=5'b00110; //7'b0000110 E
            4'b1111: reg_LEDs[6:2]=5'b11111; //7'b1111111 OFF
            default: reg_LEDs[6:2] = 5'b11111;

        endcase
    end
endmodule
```

Hex to BCD Converter

// CONVERTS HEX TO BCD

```
module hex_to_bcd_converter(input wire clk, reset,
                           input wire [19:0]
hex_number,
                           output [3:0]
bcd_digit_0,bcd_digit_1,bcd_digit_2,bcd_digit_3, bcd_digit_4, bcd_digit_5);
    // DE1-SoC has 6 7_seg_LEDs, 20 bits can represent decimal 999999.
    //This module is designed to convert a 20 bit binary representation to BCD
    integer i, k, j;
    wire [19:0] hex_number1; // the last 20 bits of input hex_number
    reg [3:0] bcd_digit [5:0]; // 6 BCD for 0-5
    assign hex_number1 = hex_number[19:0];

    assign bcd_digit_0 = bcd_digit[0];
    assign bcd_digit_1 = bcd_digit[1];
    assign bcd_digit_2 = bcd_digit[2];
    assign bcd_digit_3 = bcd_digit[3];
    assign bcd_digit_4 = bcd_digit[4];
    assign bcd_digit_5 = bcd_digit[5];

    always @ (*) begin
        if (!reset) begin
            //set all 6 digits to 0
            for (j=5 ; j>=0 ; j=j-1) begin
                bcd_digit[j] = 4'b0000;
            end
        end
        //set all 6 digits to 0
        for (j=5 ; j>=0 ; j=j-1) begin
            bcd_digit[j] = 4'b0000;
        end

        //shift 20 times
        for (i=19; i>=0; i=i-1)
            begin

                //check all 6 BCD tetrads
                for (k=5; k>=0; k=k-1)
                    begin
                        // if >=5 then add 3
                        if (bcd_digit[k] >= 5) begin
                            bcd_digit[k] = bcd_digit[k] + 4'b0011;
                        end
                    end
                end
            end
        end
    end
```

```

        end
    end

    //shift one bit of BIN/HEX left
    //for the first 5 tetrads
    for (k=5; k>=1; k=k-1)
    begin
        bcd_digit[k]=bcd_digit[k] << 1;
        bcd_digit[k][0]=bcd_digit[k-1][3];
    end

    //shift one bit of BIN/HEX left, for the last tetrad
    /* fill your code here */
    bcd_digit[0] = bcd_digit[0] << 1;
    bcd_digit[0][0] = hex_number[i];

    end //end for loop
end //end of always.
endmodule

```


PreLab 2 Top Level Module

```
module lab2_rng (
    input CLOCK_50,
    input [1:0] KEY,    // KEY[0] for Reset, KEY[1] for Resume
    output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5
);

    wire [13:0] rand_num;
    wire rnd_ready;
    wire [3:0] bcd0, bcd1, bcd2, bcd3, bcd4, bcd5;

    // random number generator
    random rng (.clk(CLOCK_50),
               .reset_n(KEY[0]),
               .resume_n(KEY[1]),
               .random(rand_num),
               .rnd_ready(rnd_ready));

    // binary to BCD converter
    hex_to_bcd_converter my_conv (.clk(CLOCK_50),

.reset(KEY[0]),

.hex_number({6'b0, rand_num}),

.bcd_digit_0(bcd0),

.bcd_digit_1(bcd1),

.bcd_digit_2(bcd2),

.bcd_digit_3(bcd3),

.bcd_digit_4(bcd4),

.bcd_digit_5(bcd5));

    // 7-segment decoder
    seven_seg_decoder h0 (.x(bcd0), .hex_LEDs(HEX0));
    seven_seg_decoder h1 (.x(bcd1), .hex_LEDs(HEX1));
    seven_seg_decoder h2 (.x(bcd2), .hex_LEDs(HEX2));
    seven_seg_decoder h3 (.x(bcd3), .hex_LEDs(HEX3));
    seven_seg_decoder h4 (.x(bcd4), .hex_LEDs(HEX4));
```

```
    seven_seg_decoder h5 (.x(bcd5), .hex_LEDs(HEX5));  
endmodule
```

Prelab 2: FSM

FSM Module

```
// check if rng is ready
always @(posedge CLOCK_50) begin
    if (rng_ready) begin
        random_wait_time = random_wait;
    end
    else begin
        random_wait_time = 2000; // default value
    end
end

// FSM
always @(posedge CLOCK_50 or negedge KEY[1]) begin
    if (!KEY[1]) begin
        curr_state <= RESET;
    end
    else begin
        curr_state <= nxt_state;
    end
end

// determine nxt_state
always @(*) begin
    // default conditions
    nxt_state = curr_state;
    sel = 2'b00;
    cheat_disp = 24'b0;

    case(curr_state)
        RESET: begin
            //disp_counter_val = 20'b0;
            sel = 2'b00;
            nxt_state = START;
            disp_counter_enable = 0;
        end

        START: begin
            sel = 2'b10; // blink
            main_counter_start = 0;
            disp_counter_reset = 0;
        end
    endcase
end
```

```

        if (ms_counter >= 5000) begin
            nxt_state = WAIT_LED;
        end
        else begin
            nxt_state = START;
        end
    end
end

WAIT_LED: begin
    sel = 2'b00; // off, waiting period
    disp_counter_reset = 0;
    // check cheating
    if (!KEY[0] && !KEY[3]) begin
        nxt_state = BOTH_CHEAT;
    end
    else if (!KEY[0]) begin
        nxt_state = P1_CHEAT;
    end
    else if (!KEY[3]) begin
        nxt_state = P2_CHEAT;
    end
    else if (ms_counter >= random_wait_time + 5000) begin
        nxt_state = COUNTING;
    end
end

// COUNTING state
COUNTING: begin
    sel = 2'b01; // display reaction time
    disp_counter_reset = 1;
    disp_counter_enable = 1; // start counting reaction
    disp_counter_start = 0;

    if (!KEY[0] && !KEY[3]) begin
        nxt_state = BOTH_CHEAT;
    end
    else if (!KEY[0]) begin
        if (disp_counter < 80) begin
            nxt_state = P1_CHEAT;
        end
        else begin
            nxt_state = P1_WIN;
        end
    end
end
end

```

```

        else if (!KEY[3]) begin
            if (disp_counter < 80) begin
                nxt_state = P2_CHEAT;
            end
            else begin
                nxt_state = P2_WIN;
            end
        end
    end
end

```

```

P1_CHEAT: begin
    sel = 2'b11;
    cheat_disp = 24'h111111;
    disp_counter_enable = 0;
    if (!KEY[2]) begin
        nxt_state = WAIT_LED;
    end
end

```

```

P2_CHEAT: begin
    sel = 2'b11;
    cheat_disp = 24'h222222;
    disp_counter_enable = 0;
    if (!KEY[2]) begin
        nxt_state = WAIT_LED;
    end
end

```

```

BOTH_CHEAT: begin
    sel = 2'b11;
    cheat_disp = 24'h888888;
    disp_counter_enable = 0;
    if (!KEY[2]) begin
        nxt_state = WAIT_LED;
    end
end

```

```

P1_WIN: begin
    disp_counter_enable = 0;
    disp_counter_start = 1;
    sel = 2'b01; // display winning time
    if (!KEY[2]) begin
        nxt_state = WAIT_LED;
    end
end

```

```

        end

        P2_WIN: begin
            disp_counter_enable = 0;
            disp_counter_start = 1;
            sel = 2'b01; // display winning time
            if (!KEY[2]) begin
                nxt_state = WAIT_LED;
            end
        end

        default: nxt_state = RESET;
    endcase
end

// update scores
always @(posedge CLOCK_50 or negedge KEY[1]) begin
    if (!KEY[1]) begin
        score1 <= 5'b0;
        score2 <= 5'b0;
    end
    else begin
        if (curr_state == P1_WIN && nxt_state == WAIT_LED) begin
            score1 <= (score1 << 1) | 5'b00001;
        end
        else if (curr_state == P2_WIN && nxt_state == WAIT_LED) begin
            score2 <= (score2 >> 1) | 5'b10000;
        end
    end
end
end

```

Mux

```
module mux4_1(input [1:0] sel,
               input [23:0] a, b, c, d,
               output reg [23:0] out);

    always@(*) begin
        case (sel)
            2'b00: out = a; // off
            2'b01: out = b; //
            2'b10: out = c;
            2'b11: out = d;
            default: out = 24'b0;
        endcase
    end
endmodule
```

Blink LEDs

```
module blink_leds (input clk,
                  input reset_n,
                  output reg [23:0] LEDs);

    parameter factor = 200;
    reg [11:0] counter;

    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            counter <= 12'b0;
            LEDs <= 24'b0;
        end

        else begin
            if (counter < factor/2) begin
                counter <= counter + 1'b1;
            end
            else if (counter < factor) begin
                counter <= counter + 1'b1;
                LEDs <= 24'b111111111111111111111111;
            end
            else begin
                counter <= 12'b0;
                LEDs <= 24'b0;
            end
        end
    end
endmodule
```